



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»
(ИУ) КАФЕДРА «Информационная безопасность» (ИУ8)

ОТЧЁТ ПО ДОМАШНЕЙ РАБОТЕ №2.2

по дисциплине «Математические основы Искусственного Интеллекта»

“Необходимые и достаточные условия существования безусловного
экстремума”

Студент: группа ИУ8-13М
Жукова Мария Андреевна
«09» октября 2025 г.

(подпись, дата)

Руководитель:
Каганов Юрий Тихонович
«09» октября 2025 г.

(подпись, дата)

Москва, 2025

СОДЕРЖАНИЕ

Цель работы.....	3
Постановка задачи	4
Основная часть.....	5
Вывод.....	8
Приложение А.....	9

Цель работы

Цель работы:

1. Исследование необходимых и достаточных условий существования экстремума функции с учетом ограничений (условный экстремум).
2. Вычисление экстремумов функции.

Постановка задачи

Дано: Дважды непрерывно дифференцируемые целевая функция $f(x)$ (ВАРИАНТ 2) и функции ограничений. Требуется исследовать функцию $f(x)$ на экстремум, т.е. определить точки $x \in R$ её локальных минимумов и максимумов на X .

$$1.2. \begin{cases} f(x) = (x_1^3 - x_2)^2 + x_2^2 \rightarrow \text{extr} \\ g_1(x) = x_1^2 + x_2^2 - 1 \leq 0, \\ g_2(x) = -x_1 \leq 0. \end{cases}$$

Шаг 1 Составить обобщенную функцию Лагранжа

Шаг 2 Записать необходимые условия экстремума первого порядка.

Шаг 3 Проверить выполнение достаточных и необходимых условий второго порядка в каждой стационарной точке двумя способами.

Шаг 4 Найти все стационарные точки и значения функций, соответствующие этим точкам.

Основная часть

$$1.2. \begin{cases} f(x) = (x_1^3 - x_2)^2 + x_2^2 \rightarrow \text{extr} \\ g_1(x) = x_1^2 + x_2^2 - 1 \leq 0, \\ g_2(x) = -x_1 \leq 0. \end{cases}$$

Рисунок 1 – Вариант 2

Найти: Все экстремумы заданной функции на интервале с учетом ограничений.

В данной части 2 домашней работы разрабатывается код нелинейного оптимизатора по следующему алгоритму:

Алгоритм решения задачи

Шаг 1. Составить обобщенную функцию Лагранжа

$$L(x, \lambda_0, \lambda) = \lambda_0 f(x) + \sum_{j=1}^p \lambda_j g_j(x)$$

Шаг 2. Записать необходимые условия экстремума первого порядка

А) $\frac{\partial L(x^e, \lambda_0^e, \lambda^e)}{\partial x_i} = 0, i = \overline{1, n};$ **Б)** $g_j(x) = 0, \quad j = \overline{1, m}; \quad g_j(x^e) \leq 0, j = m+1, \dots, p;$

В) $\lambda_j^e \geq 0, j = \overline{m+1, p}$ (для минимума), $\lambda_j^e \leq 0, j = \overline{m+1, p}$ (для максимума);

Г) $\lambda_j^e g_j(x^e) = 0, j = \overline{m+1, p}.$

Шаг 3. Решить систему уравнений для двух случаев $\lambda_0 = 0$ и $\lambda_0 \neq 0$ (при этом

$(\frac{\lambda_j^e}{\lambda_0^e} = \lambda_j^{'e})).$ В результате находится точка x^e .

Шаг 4. Для выделенных на шаге 3 точек проверить достаточные условия экстремума

$$d^2 L(x^e, \lambda^e) = \sum_{i=1}^n \sum_{j=1}^n \frac{\partial^2 L(x, \lambda)}{\partial x_i \partial x_j} dx_i dx_j > 0,$$

$$(d^2 L(x^e, \lambda^e) = \sum_{i=1}^n \sum_{j=1}^n \frac{\partial^2 L(x, \lambda)}{\partial x_i \partial x_j} dx_i dx_j < 0).$$

и $\frac{\partial L(x^e, \lambda_0^e, \lambda^e)}{\partial x_i} = 0, i = \overline{1, n}; \quad dg_j(x^e) = \sum_{i=1}^n \frac{\partial g_j}{\partial x_i} dx_i = 0, j = \overline{1, m}.$

Рисунок 2 – алгоритм работы нелинейного оптимизатора

Разработанный по приведенной на Рисунке 2 схеме код обнаружил 6 точек, являющихся решениями функции при заданных ограничениях:

```

=====
ВСЕ НАЙДЕННЫЕ ЭКСТРЕМУМЫ
=====

Локальные минимумы (2):
Минимум 1:  $x_1 = 0.000000$ ,  $x_2 = 0.000000$ ,  $f = 0.000000$ 
Минимум 2:  $x_1 = 0.000000$ ,  $x_2 = 0.000000$ ,  $f = 0.000000$ 

Локальные максимумы (6):
Максимум 1:  $x_1 = 0.000000$ ,  $x_2 = 0.000000$ ,  $f = 0.000000$ 
Максимум 2:  $x_1 = 0.000000$ ,  $x_2 = -1.000000$ ,  $f = 2.000000$ 
Максимум 3:  $x_1 = 0.000000$ ,  $x_2 = 1.000000$ ,  $f = 2.000000$ 
Максимум 4:  $x_1 = 0.000000$ ,  $x_2 = 0.000000$ ,  $f = 0.000000$ 
Максимум 5:  $x_1 = 0.000000$ ,  $x_2 = -1.000000$ ,  $f = 2.000000$ 
Максимум 6:  $x_1 = 0.000000$ ,  $x_2 = 1.000000$ ,  $f = 2.000000$ 

★ ГЛОБАЛЬНЫЙ МИНИМУМ:
 $x_1 = 0.000000$ ,  $x_2 = 0.000000$ 
 $f = 0.000000$ 

```

Рисунок 3 – значения переменных и функции

Некоторые из обнаруженных точек совпадают, различаются только значениями множителей Лагранжа. Также, были построены графики линий уровня функции с отмеченными допустимыми точками, являющимися локальными минимумами данной функции.

Линии уровня функции $f(x_1, x_2) = (x_1^3 - x_2)^2 + x_2^2$

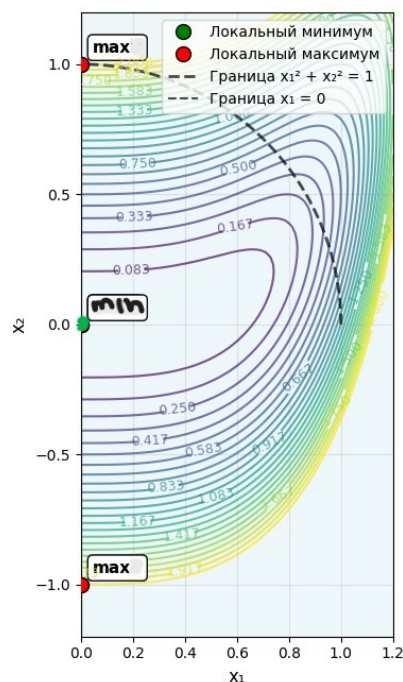


Рисунок 4 – линии уровня

3D поверхность функции

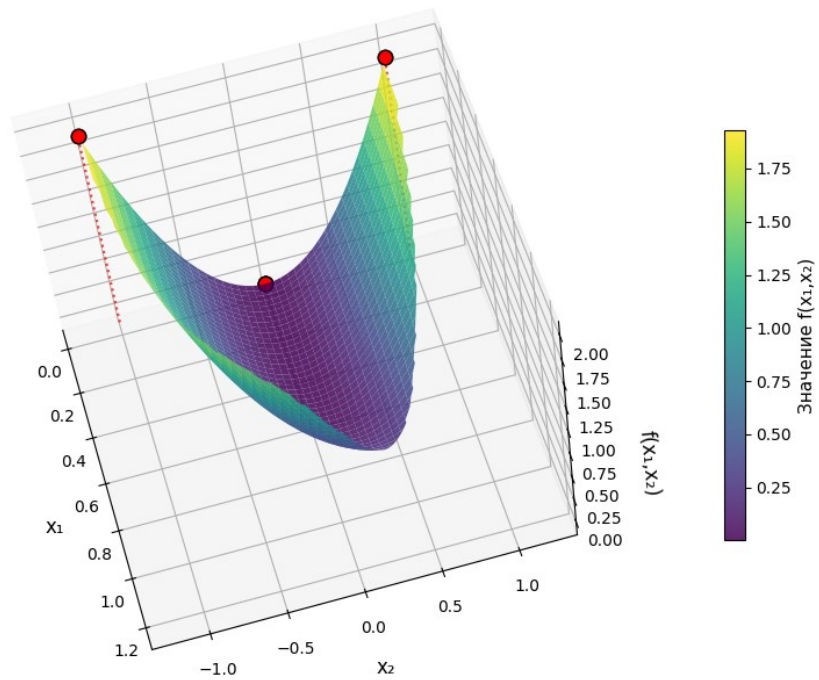


Рисунок 5 – поверхность функции

Таким образом, по методу нелинейной оптимизации с использованием множителей Лагранжа было обнаружено 3 точки, являющиеся экстремумами функции с учетом ограничений: $(-1, 0)$, $(0, 0)$, $(1, 0)$. На заданном ограничением интервале точка $(0, 0)$ является глобальным минимумом функции, а точки $(-1, 0)$ и $(1, 0)$ являются глобальными максимумами.

Вывод

Во второй части данной домашней работы было проведено Исследование необходимых и достаточных условий существования экстремума функции с учетом ограничений (условный экстремум), а также вычислены экстремумы функции и разработан код нелинейного оптимизатора методом множителей Лагранжа.

Приложение А

Код нелинейного оптимизатора. ДЗ2_2. Вариант 2.

```
'''Скрипт решает задачу нелинейной оптимизации из варианта 2 ДЗ2_2'''
import numpy as np
import matplotlib.pyplot as plt
import sympy as sp
from sympy import symbols, diff, solve

def solve_optimization():
    # Определяем переменные
    x1, x2 = symbols('x1 x2')

    # Данные Вариант 2
    f = (x1**3 - x2)**2 + x2**2
    g1 = x1**2 + x2**2 - 1 # <= 0
    g2 = -x1                # <= 0

    print("Поиск ВСЕХ локальных экстремумов")
    print("=" * 50)

    # Функция Лагранжа
    lambda0, lambda1, lambda2 = symbols('lambda0 lambda1 lambda2')
    L = lambda0 * f + lambda1 * g1 + lambda2 * g2

    solutions = []

    # Рассматриваем оба случая: минимум и максимум
    problem_types = [
        {'type': 'min', 'sign': 1, 'name': 'минимум'},
        {'type': 'max', 'sign': -1, 'name': 'максимум'}
    ]

    for problem in problem_types:
        print(f"\nПоиск {problem['name']}ов:")
        print("-" * 30)

        # Случай  $\lambda_0 = 1$ 
        L_norm = L.subs(lambda0, 1)
        dL_dx1 = diff(L_norm, x1)
        dL_dx2 = diff(L_norm, x2)

        # Рассматриваем различные комбинации активных ограничений
        cases = [
            {"name": "Никакие ограничения не активны", "active": [0, 0]},
            {"name": "Только g1 активно", "active": [1, 0]},
            {"name": "Только g2 активно", "active": [0, 1]},
            {"name": "Оба ограничения активны", "active": [1, 1]}
        ]
```

```

for case in cases:
    equations = [dL_dx1, dL_dx2]

    # Условия дополняющей нежесткости
    if case['active'][0] == 1:
        equations.append(g1)
    else:
        equations.append(lambda1)

    if case['active'][1] == 1:
        equations.append(g2)
    else:
        equations.append(lambda2)

    # Решаем систему
    try:
        sol = solve(equations, [x1, x2, lambda1, lambda2], dict=True)
        for s in sol:
            # Проверяем допустимость точки
            if (s[x1] >= -1e-10 and
                s[x1]**2 + s[x2]**2 - 1 <= 1e-10):

                # Проверяем условия на множители
                if check_multiplier_conditions(s, lambda1, lambda2,
case['active'], problem['sign']):
                    point = {x1: s[x1], x2: s[x2]}
                    f_val = f.subs(point)

                    solution_info = {
                        'point': point,
                        'f_value': f_val,
                        'multipliers': {'λ1': s.get(lambda1, 0), 'λ2':
s.get(lambda2, 0)},

                        'type': problem['type'],
                        'case': case['name']
                    }
                    solutions.append(solution_info)

                    print(f"  {case['name']}:")
                    print(f"    x1 = {float(s[x1]):.4f}, x2 =
{float(s[x2]):.4f}")
                    print(f"    f = {float(f_val):.6f}")

            except:
                continue

    return f, solutions

def check_multiplier_conditions(solution, lambda1, lambda2, activity, sign_mult):
    """Проверяет условия на множители Лагранжа"""
    # sign_mult = 1 для минимума, -1 для максимума

```

```

# Проверяем  $\lambda_1$ 
if lambda1 in solution:
    lambda_val = solution[lambda1]
    try:
        numeric_val = float(lambda_val)
        if activity[0] == 1: # Для активных ограничений
            if sign_mult * numeric_val < -1e-10:
                return False
    except:
        pass

# Проверяем  $\lambda_2$ 
if lambda2 in solution:
    lambda_val = solution[lambda2]
    try:
        numeric_val = float(lambda_val)
        if activity[1] == 1: # Для активных ограничений
            if sign_mult * numeric_val < -1e-10:
                return False
    except:
        pass

return True

def plot_solutions(f, solutions):
    # Создаем сетку для построения
    x1_vals = np.linspace(0, 1.2, 100)
    x2_vals = np.linspace(-1.2, 1.2, 100)
    X1, X2 = np.meshgrid(x1_vals, x2_vals)

    # Вычисляем значения функции на сетке
    F_vals = (X1**3 - X2)**2 + X2**2

    # Область допустимых решений
    feasible_mask = (X1 >= 0) & (X1**2 + X2**2 <= 1)

    # Создаем фигуру с двумя подграфиками
    fig = plt.figure(figsize=(16, 7))

    # ГРАФИК 1: Линии уровня
    ax1 = fig.add_subplot(121)

    # Линии уровня
    levels = np.linspace(0, 2, 25)
    contour = ax1.contour(X1, X2, F_vals, levels=levels, alpha=0.7, linewidths=1.5)
    ax1.clabel(contour, inline=True, fontsize=9)

    # Заливаем допустимую область
    ax1.contourf(X1, X2, feasible_mask, levels=1, alpha=0.2, colors=['lightblue'])

    # Отмечаем найденные точки разными цветами

```

```

minima_added = False
maxima_added = False

for i, sol in enumerate(solutions):
    x1_val = float(sol['point'][symbols('x1')])
    x2_val = float(sol['point'][symbols('x2')])

    if sol['type'] == 'min':
        marker = 'go' # Зеленый - минимум
        label = 'Локальный минимум' if not minima_added else ""
        minima_added = True
    else:
        marker = 'ro' # Красный - максимум
        label = 'Локальный максимум' if not maxima_added else ""
        maxima_added = True

    ax1.plot(x1_val, x2_val, marker, markersize=10, markeredgcolor='black',
             label=label, zorder=5)

    # Подписываем точки
    ax1.annotate(f'{"min" if sol["type"] == "min" else "max"}{i+1}',
                 (x1_val, x2_val), xytext=(8, 8),
                 textcoords='offset points', fontsize=11, fontweight='bold',
                 bbox=dict(boxstyle="round,pad=0.3", facecolor='white',
alpha=0.8))

    # Добавляем границы области
    theta = np.linspace(0, np.pi/2, 100) # Только первая четверть (x1 >= 0)
    circle_x = np.cos(theta)
    circle_y = np.sin(theta)
    ax1.plot(circle_x, circle_y, 'k--', linewidth=2, alpha=0.7, label='Граница  $x_1^2 + x_2^2 = 1$ ')
    ax1.axvline(x=0, color='k', linestyle='--', alpha=0.7, label='Граница  $x_1 = 0$ ')

    ax1.set_xlabel('x1', fontsize=12)
    ax1.set_ylabel('x2', fontsize=12)
    ax1.set_title('Линии уровня функции  $f(x_1, x_2) = (x_1^3 - x_2)^2 + x_2^2$ ', fontsize=13,
pad=15)
    ax1.grid(True, alpha=0.3)
    ax1.set_aspect('equal')
    ax1.legend(loc='upper right')
    ax1.set_xlim([0, 1.2])
    ax1.set_ylim([-1.2, 1.2])

    # ГРАФИК 2: 3D поверхность
    ax2 = fig.add_subplot(122, projection='3d')

    # Поверхность функции (только в допустимой области для лучшей видимости)
    F_masked = np.where(feasible_mask, F_vals, np.nan)
    surf = ax2.plot_surface(X1, X2, F_masked, cmap='viridis',
                           alpha=0.85, linewidth=0, antialiased=True)

```

```

# Отмечаем точки на 3D графике
for i, sol in enumerate(solutions):
    x1_val = float(sol['point'][symbols('x1')])
    x2_val = float(sol['point'][symbols('x2')])
    f_val = float(sol['f_value'])

    color = 'green' if sol['type'] == 'min' else 'red'
    ax2.scatter([x1_val], [x2_val], [f_val], color=color, s=80,
                edgecolor='black', depthshade=False, zorder=5)

    # Линии к осям для лучшей видимости
    ax2.plot([x1_val, x1_val], [x2_val, x2_val], [0, f_val],
             color=color, alpha=0.5, linestyle=':')

ax2.set_xlabel('x1', fontsize=12, labelpad=10)
ax2.set_ylabel('x2', fontsize=12, labelpad=10)
ax2.set_zlabel('f(x1,x2)', fontsize=12, labelpad=10)
ax2.set_title('3D поверхность функции', fontsize=13, pad=20)

# Настраиваем вид 3D графика
ax2.view_init(elev=30, azim=45)

# Добавляем цветовую шкалу
cbar = fig.colorbar(surf, ax=ax2, shrink=0.6, aspect=20, pad=0.1)
cbar.set_label('Значение f(x1,x2)', fontsize=11)

plt.tight_layout()
plt.show()

# Запускаем оптимизацию и строим графики
if __name__ == "__main__":
    f, solutions = solve_optimization()

    print("\n" + "="*60)
    print("ВСЕ НАЙДЕННЫЕ ЭКСТРЕМУМЫ")
    print("="*60)

    # Разделяем минимумы и максимумы
    minima = [s for s in solutions if s['type'] == 'min']
    maxima = [s for s in solutions if s['type'] == 'max']

    if minima:
        print(f"\nЛокальные минимумы ({len(minima)}):")
        for i, sol in enumerate(minima):
            x1_val = float(sol['point'][symbols('x1')])
            x2_val = float(sol['point'][symbols('x2')])
            print(f"    Минимум {i+1}: x1 = {x1_val:.6f}, x2 = {x2_val:.6f}, f = {float(sol['f_value']):.6f}")

    if maxima:

```

```

print(f"\nЛокальные максимумы ({len(maxima)}):")
for i, sol in enumerate(maxima):
    x1_val = float(sol['point'][symbols('x1')])
    x2_val = float(sol['point'][symbols('x2')])
    print(f"    Максимум {i+1}:  $x_1 = {x1\_val:.6f}$ ,  $x_2 = {x2\_val:.6f}$ ,  $f = {float(sol['f\_value']):.6f}$ ")

# Глобальные экстремумы
if minima:
    global_min = min(minima, key=lambda x: x['f_value'])
    x1_val = float(global_min['point'][symbols('x1')])
    x2_val = float(global_min['point'][symbols('x2')])
    print(f"\n★ ГЛОБАЛЬНЫЙ МИНИМУМ:")
    print(f"     $x_1 = {x1\_val:.6f}$ ,  $x_2 = {x2\_val:.6f}$ ")
    print(f"     $f = {float(global\_min['f\_value']):.6f}$ ")

if maxima:
    global_max = max(maxima, key=lambda x: x['f_value'])
    x1_val = float(global_max['point'][symbols('x1')])
    x2_val = float(global_max['point'][symbols('x2')])
    print(f"\n★ ГЛОБАЛЬНЫЙ МАКСИМУМ:")
    print(f"     $x_1 = {x1\_val:.6f}$ ,  $x_2 = {x2\_val:.6f}$ ")
    print(f"     $f = {float(global\_max['f\_value']):.6f}$ ")

# Строим графики
plot_solutions(f, solutions)

```